Planner-Distilled Decision Transformers Break the Behaviour-Cloning Ceiling in Multi-Market BESS Dispatch

Hou Nam U (Victor)
github.com/mrvictoru/energydecision

Abstract

Grid-scale batteries in Australia's National Electricity Market must co-optimise energy arbitrage with eight frequency-control ancillary service (FCAS) markets while paying for battery wear. We explore whether an offline sequence model — a Decision Transformer — can act as a single, generalising controller for this multi-market dispatch problem: learning from logged data rather than risky online exploration, and optimising revenue net of degradation. We first train it on trajectories generated by online reinforcement-learning policies (PPO and other actor–critic agents). That route stalls at a *behaviour-cloning ceiling*: the logged policies bid little into FCAS, and none of the tested changes to data scale or curation, return-to-go prompting, loss re-weighting, or online fine-tuning lets the Decision Transformer exceed the FCAS behaviour present in its logs. We therefore change the data source rather than the learner, distilling an honest, non-clairvoyant stochastic-dynamic-programming planning teacher into a standalone Decision Transformer and conditioning it on a state-dependent return-to-go prompt derived from the planner's cost-to-go. The resulting policy beats a tuned PPO baseline on four held-out surfaces (standard, dispatch-matched, expanded broad-2024, and 2025 out-of-distribution) by $3.84\times$, $1.80\times$, $6.70\times$, and $6.39\times$, with all four paired-difference 95% confidence intervals excluding zero, and passes a market-impact gate on all nine grid-scale cells with no solver in the control loop. Results are simulator-based; sim-to-real transfer is the main open question.

1 Introduction

Grid-scale batteries are now a material part of Australia's National Electricity Market (NEM), earning revenue both from energy arbitrage and from Frequency Control Ancillary Services (FCAS), eight reserve markets that a single controller must bid into simultaneously. Every five minutes the controller chooses a nine-dimensional action: how much to charge or discharge, and how much capacity to offer into each reserve service, while accounting for the wear each cycle imposes on the battery. This is a sequential decision problem under uncertainty, and it is in principle well suited to reinforcement learning [Subramanya et al., 2022].

The practical obstacle is that online reinforcement learning learns by exploration. In a live market, exploration is expensive, risky, and slow to converge, and the value is concentrated in rare events (energy price spikes and FCAS scarcity prices) that a learner must experience to price. Learning from logged data avoids exploration entirely. The Transformer established attention-based sequence modelling as a practical architecture, and autoregressive Transformer language models later showed broad few-shot performance across language tasks [Vaswani et al., 2017, Brown et al., 2020]. This versatility motivates exploring sequence models for control, although success in language does not establish success in a battery market. The Decision Transformer (DT) adapts this sequence-modelling approach to offline RL: it is trained on fixed trajectories with no environment interaction, and it

Preprint.

conditions its actions on a return-to-go (RTG) prompt describing how much value should remain [Chen et al., 2021]. Offline learning, however, inherits the behaviour of its data: return conditioning can push the policy above the returns in its logs — the original work reports both gains and regressions when the target is set beyond the data — but the behaviour it can express is still assembled from what those trajectories contain.

We set out to test whether an offline sequence model can serve as a single, generalising controller for this problem — one that learns from logged data, needs no online exploration, and optimises revenue net of degradation. We begin with the most natural data source, trajectories generated by online reinforcement-learning policies (PPO and other actor–critic agents), and that route stalls at a *behaviour-cloning ceiling* (Figure 1): across a systematic search within the cloning paradigm (larger and better-curated offline datasets; RTG prompting; loss re-weighting; mixed action heads; architectural changes; and online fine-tuning), none of the tested approaches lets the DT exceed the FCAS-bidding behaviour present in its logged data, and it stays well below a tuned proximal-policy-optimisation (PPO) baseline on broad and out-of-distribution surfaces. In these experiments, the ceiling tracked the data source more than the tested model and optimisation choices. Our search ranged wider than the routes reported in full here; Appendix B lists the datasets and planner variants we explored and set aside, with the reason in each case.

Our contributions are threefold:

- **Diagnosis.** We isolate the behaviour-cloning ceiling and show that none of the tested within-cloning levers removes it (Figure 1; Appendix A).
- **Method.** We break it by distilling an *honest*, non-clairvoyant stochastic-dynamic-programming (SDP) planning teacher [Abdulla et al., 2018] into a standalone DT and conditioning the student on a *state-dependent* RTG prompt derived from the planner's cost-to-go $J_t(\mathrm{soc})$ (Section 4). No solver runs in the control loop at deployment.
- **Evaluation.** We introduce a market-impact gate that re-prices the battery's own dispatch endogenously, and evaluate a prompt fallback under market impact: the shipped policy passes the gate on all nine grid-scale cells (Section 6).

On four held-out surfaces, the standalone DT beats tuned PPO by 3.84× on standard 2024, 1.80× on dispatch-matched, 6.70× on expanded broad-2024, and 6.39× on 2025 out-of-distribution data, with all four paired-difference 95% confidence intervals excluding zero (Section 5); relative to its solver-in-the-loop teacher on the same surfaces it retains 77% of the teacher's aggregate profit on standard 2024 and 70% on dispatch-matched, and *exceeds* it on expanded broad-2024 (132%) and 2025 out-of-distribution (276%), where the teacher's seasonal-forecast planning generalises less well, all while removing the solver from the control loop. All results are simulator-based and driven by historical NEM data; sim-to-real transfer is the main open question. Code, trajectory corpora, and model weights are released with this paper.

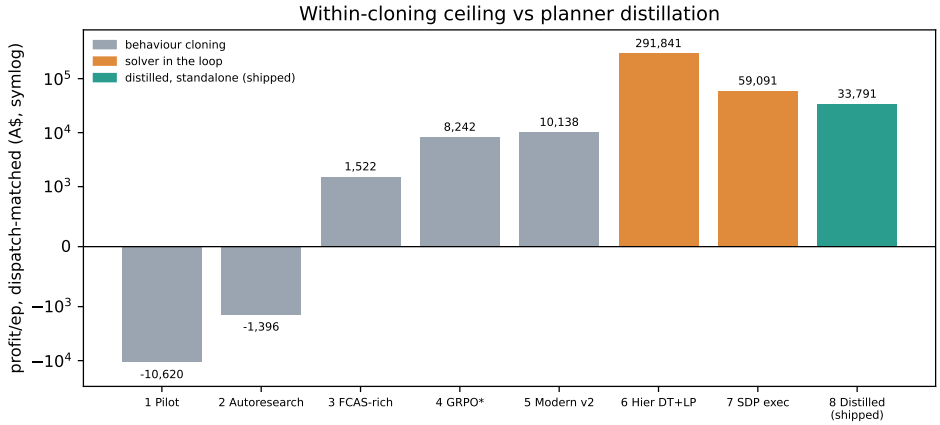


Figure 1: The behaviour-cloning ceiling and how it is broken. Within the cloning paradigm (grey) profit improves then plateaus; solver-in-the-loop planners (orange) exceed it but are not deployable; the distilled standalone student (teal) keeps the ceiling broken with no solver in the control loop. Values are per-stage evaluation surfaces.

2 Related Work

Sequence modelling reframes control as autoregressive prediction over (return, state, action) tokens. The Decision Transformer conditions on a target return-to-go and predicts actions, matching or exceeding state-of-the-art offline RL baselines on several offline benchmarks [Chen et al., 2021]; related work applies the recipe online [Zheng et al., 2022], to value-based relabeling [Yamagata et al., 2023], and to whole-trajectory modelling [Janner et al., 2021]. We adopt the Decision Transformer, derive its RTG prompt from a planner's cost-to-go, and evaluate a surface-aware prompt fallback under market impact.

Offline value-based RL instead guards against over-estimating unseen actions: conservative Q-learning penalises Q-values under selected action distributions; its analysis gives a lower bound on expected policy value [Kumar et al., 2020], while implicit Q-learning avoids querying unseen actions at all, fitting an upper expectile instead [Kostrikov et al., 2022]. These are the natural alternatives to sequence modelling; we chose the Decision Transformer for its inference-time RTG conditioning, and leave a direct IQL/CQL comparison to future work.

DAgger is a related imitation-learning method: it iteratively queries an expert on states visited by the student, adds those corrections to the training set, and re-trains the student [Ross et al., 2011]. Our approach instead trains the DT on trajectories generated by the planning teacher; it does not perform DAgger-style interactive aggregation. The teacher is an honest, non-clairvoyant stochastic-dynamic-programming planner over a seasonal price forecast [Abdulla et al., 2018]. The teacher does not use realised future prices. Our 2025 OOD results evaluate the resulting policy, but do not isolate the effect of this information restriction against a student distilled from a perfect-foresight teacher.

Battery scheduling is a mature application area: dynamic-programming and model-predictive approaches optimise operation against forecasts and degradation [Abdulla et al., 2018], and detailed aging models quantify the wear those schedules induce [Muenzel et al., 2015, Kampker et al., 2025]; reinforcement learning has been applied to storage in a range of settings [Subramanya et al., 2022]. Our setting differs in scope: a nine-dimensional co-optimisation over energy and eight FCAS services, evaluated across multiple held-out surfaces with an explicit market-impact gate.

3 Experimental Framework

3.1 Market environment

We model a grid-scale battery participating in Australia's National Electricity Market (NEM), using the `AEMOBatteryTradingEnv` environment. The environment advances in five-minute intervals and

exposes an 18-dimensional observation: calendar features (hour-of-day and day-of-year sine/cosine encodings and a peak-period indicator), the regional reference price (RRP), total regional demand, the eight frequency-control ancillary service (FCAS) prices, the solar and wind generation fractions, and the battery state of charge. The action is nine-dimensional: one energy-dispatch command in $[-1, 1]$ (positive charges, negative discharges) and eight FCAS bids in $[0, 1]$, one per service (raise and lower regulation, and the 6-second, 60-second, and 5-minute contingency services). The reward is the per-step profit in dollars: energy revenue plus FCAS revenue minus degradation cost, with a small penalty for any state-of-charge-limit violation.

3.2 Battery and degradation model

Each battery is specified by an energy capacity, a power limit, and an initial state of charge. Training uses four configurations spanning 2–50 MWh, at a constant cell temperature of 30°C. Battery wear is modelled with a real-world calendar-and-cycle aging model: calendar aging from time- and temperature-driven solid-electrolyte-interphase growth (Arrhenius temperature dependence), and cycle aging that grows as a power law in depth of discharge and increases with the C-rate (charge/discharge rate). We use the lithium-iron-phosphate (LFP) preset; the combined calendar-plus-cycle formulation follows Kampker et al. [2025], and cycles are identified by rainflow counting. The earlier multi-factor cycle-life model of Muenzel et al. [2015] is also implemented in the released code but is not used for the results reported here. Wear is charged to the policy as part of the environment reward and reported as a cost per unit of throughput.

3.3 Market-impact model

By default the battery is a price taker: it trades at the observed historical RRP and FCAS prices, which its own dispatch does not move. For impact-aware evaluation we use a piecewise-merit-order model in which the battery's dispatch changes the realised prices (charging adds demand and raises the energy price, discharging adds supply and lowers it), and each FCAS price is reduced in proportion to the battery's share of an estimated service depth. This is a per-interval approximation of an endogenous market, not a full market re-clear. We call evaluations without this model *identity* surfaces and those with it *impact* surfaces; Section 6 reports the impact gate.

3.4 Data

The simulator is driven by historical AEMO market data: dispatch prices for energy and the eight FCAS services, regional demand, and generation by fuel type. Training trajectories and the seasonal price profile are drawn from pre-2024 windows; evaluation uses 2024 (identity surfaces) and 2025 (out-of-distribution) windows that are held out by date.

4 Method

4.1 SDP planner as teacher

Dynamic programming solves a finite-horizon control problem by working backward from a terminal cost. At each time and state, it selects a feasible action that minimises the immediate cost plus the cost-to-go of the resulting state. In battery dispatch, the state includes the battery's state of charge (SOC), and the actions control charging or discharging. The general battery-planning formulation is related to prior work on SDP-based storage scheduling [Abdulla et al., 2018]; here, we describe the specific non-clairvoyant teacher used to generate our trajectories.

For energy dispatch, the teacher constructs a regional reference price (RRP) forecast by averaging historical RRP within month-of-year and hour-of-day bins. The profile is estimated from pre-2024 processed data and indexed by the settlement times in each episode. The energy planner therefore uses seasonal mean prices, not the episode's realised future RRP. FCAS is handled separately: the bidding rule uses prices available at the current step and does not forecast future FCAS prices. We use "non-clairvoyant" to describe this restricted information set, in contrast to a planner given realised future prices. For example, the NSW1 October seasonal profile gives mean RRP values of about A\$77/MWh at 07:00 and A\$184/MWh at 18:00. The teacher therefore expects lower-priced

morning energy and higher-priced evening energy, while actual prices on any particular day may differ substantially from those historical means.

The teacher is a three-stage pipeline (Figure 2):

1. **SOC waypoints.** A learned model proposes a sequence of intermediate SOC targets. It supplies targets to the planner but does not optimise the dispatch actions.
2. **Energy dispatch.** For each interval between waypoints, a discretised dynamic program performs backward induction over SOC states and candidate charge/discharge actions. It minimises forecast energy cost plus estimated degradation cost, with a soft terminal penalty that encourages the final SOC to approach the interval's target. The wear estimate combines the planner's degradation estimator with a linear throughput surrogate; the estimator is scaled by `deg_calibration`.
3. **FCAS bidding.** Given the energy action, a greedy rule allocates residual raise and lower headroom to the highest-priced eligible service in each direction, using current FCAS prices.

Thus, the teacher combines learned waypoint selection, dynamic-programming energy planning, and rule-based FCAS bidding. It is not a pure SDP solver or a joint optimiser of energy and FCAS.

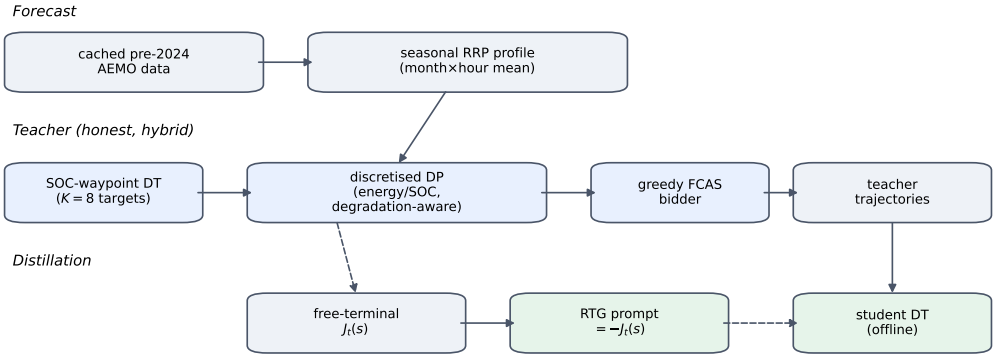


Figure 2: The planner-distillation pipeline. A non-clairvoyant, hybrid teacher (seasonal forecast; SOC-waypoint model; degradation-aware dynamic program; greedy FCAS bidder) generates trajectories for offline Decision-Transformer training. Separately, the free-terminal cost-to-go $J_t(s)$ supplies the state-dependent RTG prompt at inference (dashed). No solver runs in the control loop; the value table is precomputed.

We also compute a free-terminal cost-to-go table $J_t(s)$ over time t and discretised SOC s:

$$J_t(s) \;=\; \min_{a \in \mathcal{A}(s)} \left[c_t(s,a) + J_{t+1}(s') \right], \qquad J_T(s) = 0, \tag{1}$$

where $\mathcal{A}(s)$ is the set of feasible actions at SOC s, s' is the resulting next SOC, and c_t includes forecast energy cost and the degradation surrogate. Unlike the waypoint-constrained dispatch plan, this table has no terminal SOC target. At inference, the table is queried at the current time and SOC, and the Decision Transformer receives $-J_t(s)$ as its return-to-go prompt. The table provides a value-conditioning signal, not the SDP policy: the student selects actions in a single forward pass, with no solver in the control loop. The table itself is built by the same seasonal dynamic program; because it uses no realised prices, it can be precomputed and cached per region, season, and battery.

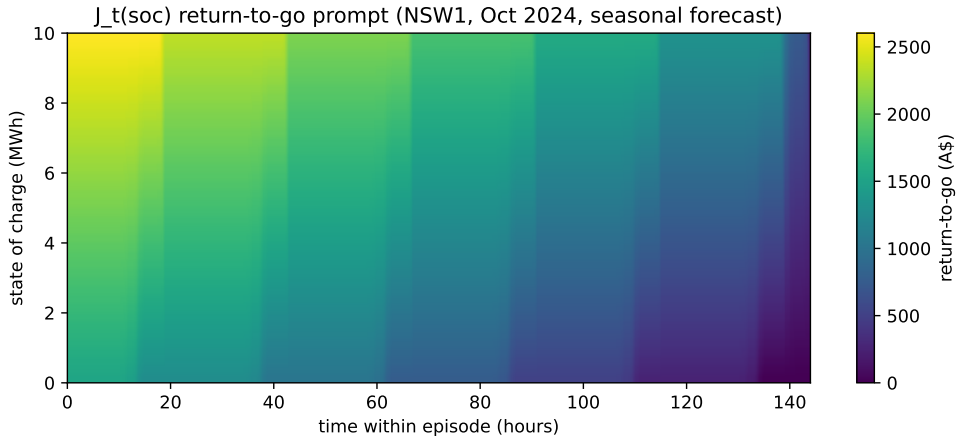


Figure 3: The $J_t(\mathrm{soc})$ return-to-go prompt for a representative window (NSW1, October 2024, seasonal forecast). Value rises with stored energy and decays as the remaining episode shortens; this table supplies the DT's state-dependent RTG prompt.

4.2 Trajectory corpus

We generated the student-training corpus by sampling random, fixed-length windows from cached, preprocessed five-minute AEMO data and rolling out the hybrid teacher described above. The corpus spans five NEM regions, two episode horizons (3,456 steps, or 12 days; and 16,128 steps, or 8 weeks), and four battery configurations, with eight episodes per region–horizon–battery combination. This yields 320 episodes. The shipped conservative, wear-calibrated corpus contains approximately 3.13 million per-step rows. Each episode is generated by running the waypoint model, SDP energy planner, and greedy FCAS bidder in the environment, so its observations, teacher actions, and rewards come from the same rollout rather than being assembled from separate sources [U, 2026].

Each row stores the normalised 18-dimensional observation, the 9-dimensional action (one energy-dispatch component and eight FCAS bids), and the resulting per-step reward, together with episode and step identifiers and metadata. For the shipped corpus, a per-step `rtg_value` is also provided: it is computed as $-J_t(s)$ from the free-terminal cost-to-go table, not as a logged discounted sum of future rewards. The exported rows do not retain timestamps. Accordingly, temporal non-overlap between training and evaluation was checked against the source data windows, rather than by auditing timestamps stored in each corpus row.

4.3 Student Decision Transformer

The student is a causal Decision Transformer trained on the teacher-generated sequences described above. At each step, it embeds the return-to-go, normalised observation, and action as an ordered sequence and uses the preceding context to predict actions. The shipped model has 8 Transformer blocks with hidden dimension 768, grouped-query attention with 12 query heads and 6 key–value heads, QK-Norm, rotary positional embeddings, and SwiGLU feed-forward layers. Its context covers 210 time steps. A mixed action head applies a hyperbolic tangent to the energy-dispatch output, which lies in $[-1, 1]$, and a sigmoid to the eight FCAS-bid outputs, which lie in $[0, 1]$. Training minimises a weighted combination of action, state, and return prediction losses:

$$\mathcal{L} = w_a \,\mathrm{MSE}_a + w_s \,\mathrm{MSE}_s + w_r \,\mathrm{MSE}_r, \qquad (w_a, w_s, w_r) = (0.999,\ 0.002,\ 0.0001). \tag{2}$$

The loss is therefore dominated by action prediction, while state and return prediction provide lower-weight auxiliary objectives. To keep the RTG inputs numerically scaled for training, we use an automatically calibrated `return_scale`; for the shipped checkpoint it is 23,284.83, which maps return-to-go values to order-10 magnitudes for the network.

4.4 Return-to-go prompting and `auto` mode

At inference time, the Decision Transformer is conditioned on a return-to-go (RTG) prompt derived from the teacher's free-terminal value table. Specifically, we set $\mathrm{RTG} = -J_t(s)$, normalised by the checkpoint's `return_scale`, so that higher remaining value corresponds to a larger prompt token. This turns the SDP cost-to-go into a value-conditioning signal while keeping the control loop solver-free: the student selects actions with a single forward pass. The value table is produced by a single backward-induction pass per episode, not per decision, and because it is a function only of the seasonal forecast and the SOC grid (no realised prices), it can be precomputed offline and cached per region, season, and battery, leaving only a constant-time lookup at each step. Figure 3 shows the table for a representative window.

For price-taking evaluation surfaces, the shipped inference mode `auto` resolves to the state-dependent `j_t_soc` prompt: the agent looks up $J_t(s)$ for the current time and SOC and feeds its negative, scaled by `return_scale`, to the transformer as RTG. Under a market-impact model (Section 6), where the battery's own dispatch moves prices, `auto` instead falls back to a constant `rtg0.0` prompt. This is a robustness choice, not a failure mode: on large batteries, a value prompt that is too optimistic can over-trade energy and self-suppress realised prices, so the conservative constant prompt is used once the battery is no longer a negligible price taker. Using the state-dependent prompt directly in this regime, rather than falling back to a constant, remains an open item.

5 Experiments

We evaluate the student on four held-out surfaces. The *standard 2024* surface contains six periods from each of five regions ($n = 30$ scenario cells). The *dispatch-matched* surface covers all twelve months of SA1 2024 ($n = 12$). The *expanded broad-2024* surface spans five regions and six periods ($n = 30$). Finally, the *2025 OOD* surface evaluates six NSW1, QLD1, and SA1 scenario cells from January and February ($n = 6$). Training data and the seasonal price profile use pre-2024 source windows; the evaluation windows are held out by date.

We compare the Decision Transformer with a tuned PPO policy, an `fcas_rule` policy, and dispatch replay (the latter only on the dispatch-matched surface). PPO is tuned using an Optuna search over ten trials and trained for up to four million environment steps. Evaluation uses five-minute steps and one rollout per scenario–battery cell. PPO and rule-policy reference rollouts are run against the same evaluation scenarios as the student, with fresh reference caches to keep the comparisons consistent.

We report mean episode profit and paired differences between the student and PPO. Uncertainty in the paired differences is estimated by bootstrap resampling the matched scenario cells with 10,000 replicates; paired Wilcoxon tests are reported as a complementary analysis. We treat the paired-difference confidence interval as the primary inferential result. Scenario cells from the same region may be correlated, so the cell count should not be interpreted as an equal number of independent regions. The student is deterministic at evaluation time; baseline reference rollouts can vary slightly run-to-run (see Table 1).

5.1 Headline Results

Table 1 summarises the extended identity-surface results. Figure 4 shows the per-surface comparison and the paired differences, and Figure 5 decomposes profit by revenue source. The student has higher mean profit than PPO on all four surfaces, and each paired-difference 95% confidence interval excludes zero. Relative to its solver-in-the-loop teacher on the same surfaces, the student retains 77% of the teacher's aggregate profit on standard 2024 and 70% on dispatch-matched, and exceeds it on expanded broad-2024 (132%) and 2025 out-of-distribution (276%), where the teacher's seasonal-forecast planning generalises less well; the per-cell ratios are noisier than these aggregates. The extended surfaces also evaluate more windows than the earlier small-sample runs, while retaining a small sample on the 2025 OOD surface.

Table 1: Extended identity surfaces (shipped physics-v2 checkpoint, `rtg_mode="auto"`). **All four paired-difference 95% CIs exclude zero.** Baselines may show ∼1% run-to-run variation.

Surface	n	DT ($/ep)	PPO ($/ep)	Δ (DT−PPO)	95% CI on Δ
Standard 2024	30	16,501	4,303	+12,198	[9,181, 14,738]
Dispatch-matched	12	33,791	18,818	+14,973	[4,794, 22,663]
Expanded broad-2024	30	29,124	4,350	+24,775	[13,509, 41,434]
2025 OOD	6	30,791	4,817	+25,974	[12,708, 49,850]

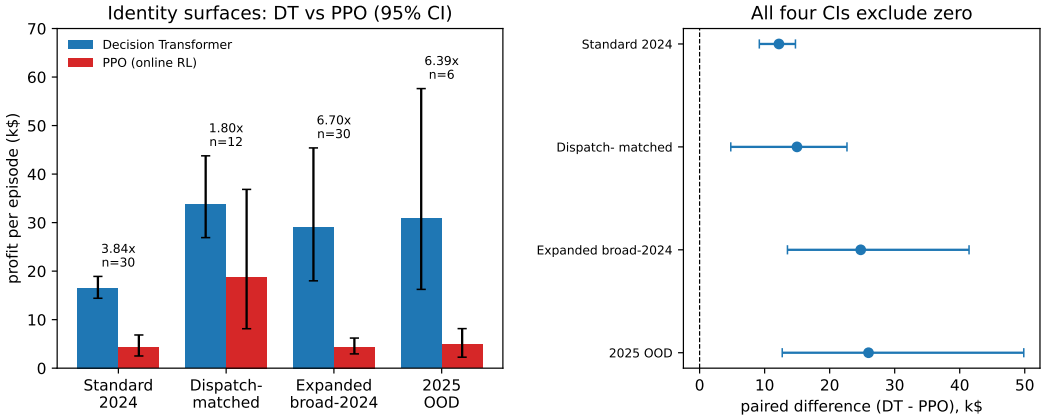


Figure 4: Decision Transformer vs PPO across the four extended identity surfaces. Left: per-episode profit with 95% bootstrap CIs. Right: the paired difference (DT − PPO) with its 95% CI; all four exclude zero.

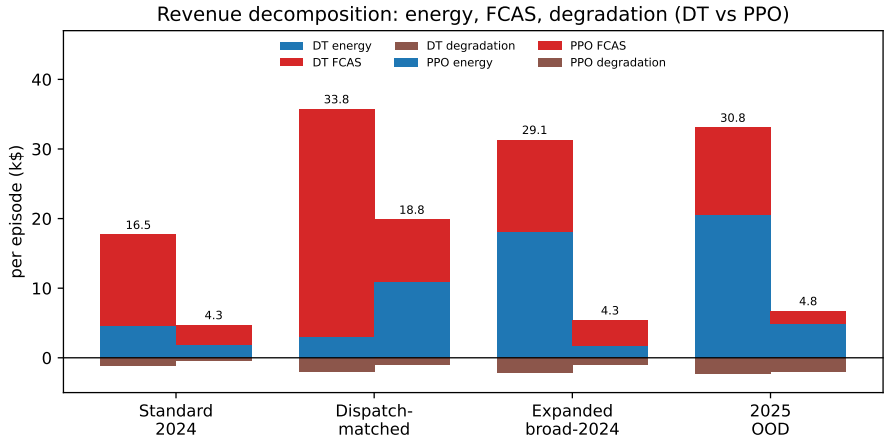


Figure 5: Revenue decomposition (energy, FCAS, degradation) per surface for the DT (solid) and PPO (hatched); the annotated number is net profit per episode. The DT's advantage is dominated by FCAS capture.

6 Market-Impact Gate

To test robustness to endogenous prices we use the piecewise-merit-order model of Section 3: energy prices shift with the battery's net dispatch (charging raises demand, discharging raises supply), and FCAS prices are attenuated by the battery's share of estimated service depth. This is a per-interval approximation of price impact, not a full market re-clear.

The gate comprises nine matched scenario–battery cells: three grid-scale batteries (small, Hornsdale-class, and Torrens Island-class) in three scenarios (SA1 October and November, VIC1 October). Under market impact, `auto` uses the fixed `rtg0.0` prompt rather than the state-dependent $J_t(s)$ prompt, and we compare this shipped fallback with PPO on the same cells. The Decision Transformer earns more than PPO in all nine cells. Across the nine paired differences, its mean advantage is \$167,099 per episode (95% bootstrap confidence interval [\$101,213, \$227,362]). At $n = 9$ the two-sided Wilcoxon minimum is 0.0039, which the paired test attains; the bootstrap interval is the primary evidence. The ratio of mean profits is 4.36× for the small battery, 4.70× for Hornsdale, and 4.67× for Torrens Island. For the price-taking counterpart, see the identity-surface comparisons in Section 5.

These results support a mode-selection rule, not a claim that one prompt is best in every regime: use the state-dependent value prompt when the battery is approximately price-taking, and the conservative constant prompt when its dispatch can materially affect prices. In this sense, market power is a gate on value-based prompting. Figure 6 summarises the gate.

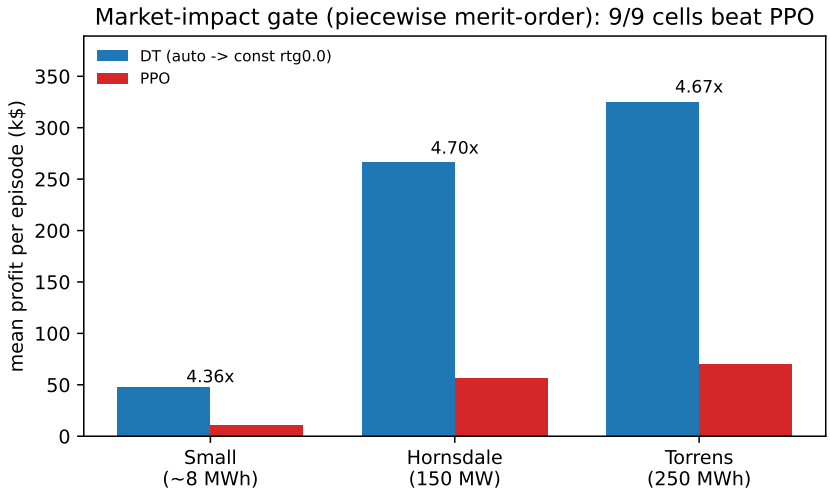


Figure 6: Market-impact gate (piecewise merit-order). Mean profit per episode for the shipped DT (fixed `rtg0.0` fallback) vs PPO across three grid-scale batteries; the DT wins all 9 scenario×battery cells.

7 Limitations

Simulator-based. All results come from our simulator, driven by historical NEM data; sim-to-real transfer is the main open item. The market-impact model is a per-interval merit-order approximation, not a full market re-clear.

Same-simulator teacher and evaluation. The teacher and the evaluation share the simulator, so the student may inherit environment-specific behaviour. Held-out 2025 data and the endogenous-price impact gate mitigate this, but the circularity is reduced, not removed.

Baseline strength and noise. The PPO reference is tuned (Optuna, up to four million steps), but its rollouts vary slightly run-to-run at long horizons (∼1%), whereas the student is deterministic. We therefore treat the paired-difference confidence interval as the primary evidence and report cell counts honestly.

Sample size. Some surfaces have few cells (as few as six), and cells from the same region are correlated, so the cell count overstates the number of independent regions. The paired-difference bootstrap is the primary inferential tool; the Wilcoxon test is indicative at these sizes.

Sensitivity to physics. Correcting the shared degradation and efficiency models moved the headline numbers substantially. The shipped results use the corrected physics; several earlier numbers are superseded and are not relied upon here.

Retention and scope. Distillation is not uniformly lossy. The student retains 70–77% of its solver-in-the-loop teacher's aggregate profit on the in-distribution surfaces (standard, dispatch-matched) and *exceeds* it on the broad and out-of-distribution surfaces, where the teacher's seasonal-forecast planning generalises less well. Any single retention figure is therefore surface-dependent, and the per-cell ratios are noisier than the aggregates. The approach also does not transfer automatically to the residential scale, where the same recipe over-cycles on long horizons.

Provenance. The shipped checkpoint's `return_scale` (23,284.83) is the correct value for inference, but it does not reproduce exactly as $\mathrm{p90}(|J_t|)/10$ of the released corpus, suggesting the released corpus may differ slightly from the file used at training; the discrepancy is documented in the released audit notes.

8 Conclusion

We set out to test whether an offline sequence model could serve as a single, generalising controller for multi-market battery dispatch, learning from logged data without online exploration and optimising revenue net of degradation. Training on trajectories from online reinforcement-learning policies stalled at a behaviour-cloning ceiling: none of the tested approaches within the cloning paradigm exceeded the FCAS-bidding behaviour present in the logged data (C1). In these experiments, the ceiling tracked the data source more than the tested model choices. We broke it by distilling an honest, non-clairvoyant SDP planning teacher into a standalone Decision Transformer prompted with a state-dependent cost-to-go $J_t(\mathrm{soc})$ (C2), so the student roughly matches the teacher's aggregate profit on the in-distribution surfaces and exceeds it out-of-distribution, with no solver in the control loop. We then evaluated prompt selection under market impact: the shipped surface-aware policy passes a piecewise merit-order impact gate on all nine grid-scale cells (C3).

Overall, the results argue that offline sequence modelling is a credible control prior for utility-scale battery trading when the data source is chosen well — most of the planner, none of the solver in the control loop. The main open question is sim-to-real transfer, which we leave to a shadow-mode deployment and to extending the market-impact model beyond its per-interval approximation. A further open direction is corpus composition: we tested re-weighting and re-composition *within* the behaviour-cloning corpora (Appendix A), but not a mixture of teacher-generated and policy-generated trajectories, which could combine the teacher's FCAS bidding with the policies' energy behaviour. We also plan controlled comparisons of selected newer sequence-model architectures against the shipped 8-block, 768-dimensional backbone, holding the data and evaluation protocol fixed. This paper does not establish that the current backbone is architecture-optimal.

Acknowledgments and Disclosure of Funding

Use of AI tools. AI coding assistants were used during development and debugging of the research codebase, and AI writing assistants were used to draft and revise portions of this manuscript. All experiments, analyses, numerical results, and citations were verified by the author, who takes full responsibility for the content.

References

Khalid Abdulla, Julian De Hoog, Valentin Muenzel, Frank Suits, Ken Steer, Andrew Wirth, and Saman Halgamuge. Optimal operation of energy storage systems considering forecasts and battery degradation. *IEEE Transactions on Smart Grid*, 9(3):2086–2096, 2018. doi: 10.1109/TSG.2016.2606490.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020. arXiv:2005.14165.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2106.01345.

Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2106.02039.

Achim Kampker, Benedikt Späth, Xiaoxuan Song, and Datao Wang. Modelling of battery energy storage systems under real-world applications and conditions. *Batteries*, 11(11):392, 2025. doi: 10.3390/batteries11110392.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2110.06169.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2006.04779.

Valentin Muenzel, Julian De Hoog, Marcus Brazil, Arun Vishwanath, and Saman Kalyanaraman. A multi-factor battery cycle life prediction methodology for optimal battery management. In *Proceedings of the 2015 ACM Sixth International Conference on Future Energy Systems (e-Energy '15)*, 2015. doi: 10.1145/2768510.2768532.

Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. arXiv:1011.0686.

Rakshith Subramanya, Seppo A. Sierla, and Valeriy Vyatkin. Exploiting battery storages with reinforcement learning: A review for energy professionals. *IEEE Access*, 10:54484–54506, 2022. doi: 10.1109/ACCESS.2022.3176446.

Hou Nam U. energydecision: a degradation-aware aemo/nem battery-control research repository. `https://github.com/mrvictoru/energydecision`, 2026. Trajectory generator: `scripts/generate_sdp_dt_trajectories.py`.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.

Taku Yamagata, Ahmed Khalil, and Raul Santos-Rodriguez. Q-learning decision transformer: Leveraging dynamic programming for conditional sequence modelling in offline rl. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2209.03993.

Qinqing Zheng, Amy Zhang, and Aditya Grover. Online decision transformer. In *International Conference on Machine Learning (ICML)*, 2022. arXiv:2202.05607.

A Within-Cloning Levers that Did Not Break the Ceiling

Figure 1 shows that the ceiling breaks only when the training data changes. Table 2 lists the levers we varied within the behaviour-cloning paradigm; none removes it, across architectures and across single-month and broad surfaces. All are evaluated at five-minute resolution on the broad (expanded-2024) surface unless noted.

Table 2: Within-cloning levers and their outcomes. None of the tested levers closes the FCAS-bidding gap to PPO; within this tested set, outcomes track the logged-data composition.

Lever	Outcome
RTG prompt sweep (0–100)	Flat — not a prompting artifact
PPO-only training data (legacy and modern architectures)	Energy-heavy ($\sim$\$17.6–17.8k profit, $\sim$\$2.1k FCAS); architecture is not the determinant
FCAS-heavy-policy subset (A2C/TD3/SAC/DDPG episodes)	FCAS capture $+23\%$ (\$4.8k $\rightarrow$ \$5.9k) but still $1.7\times$ below PPO
FCAS-weighted action loss	No effect — no higher-FCAS behaviour in the data to amplify
Mixed action head (sigmoid FCAS dims)	Ambiguous: helps 2025 OOD ($+22\%$), hurts in-distribution
Online fine-tuning (GRPO; full-PPO value critic)	Flat-to-negative; the legacy GRPO champion was a narrow overfit (\$8,242 $\rightarrow$ \$1,533)
Explicit forecast tokens (48-step TTM)	Negative: \$4,564 vs \$4,991 for the modern v2 baseline

These results suggest that, in the tested settings, the ceiling is driven chiefly by the logged data: the FCAS-bidding behaviour the student must learn is poorly represented, and none of the tested prompting, re-weighting, or fine-tuning changes supplied it. Section 4 addresses this limitation by changing the data source.

B Datasets and Routes Explored

The shipped recipe is the survivor of a wider search over data sources and planner variants. Table 3 records what we tried, why, and how each route was resolved. Two patterns stand out. First, the tested behaviour-cloning routes — the FCAS-rich, PPO-only, and FCAS-heavy corpora — all remained limited in FCAS bidding despite changes in dataset composition. This suggests that the logged policies' FCAS behaviour, rather than corpus size alone, was a key limitation. Second, the first teacher we built was not deployable (it was given realised future prices), which motivated the honest, non-clairvoyant planner of Section 4.

Table 3: Datasets and routes explored, and how each was resolved.

Route	Purpose	Outcome / disposition
Residential benchmark (`SolarBatteryEnv`, Ausgrid)	Legacy household benchmark	kWh-scale, single-tariff market; out of scope here
Original grid-scale corpus (162 eps, 5 regions)	First AEMO corpus	Superseded by the richer corpora below
FCAS-rich corpus (2,425 eps: PPO, TD3, A2C, DDPG, SAC, rule-based)	Behaviour cloning	Stalls at the ceiling (Appendix A)
PPO-only corpus (900 eps)	Does data composition explain the ceiling?	Energy-heavy, no FCAS edge; negative
FCAS-heavy subset (1,200 eps: A2C/TD3/SAC/DDPG)	Does more FCAS data help?	FCAS capture $+23\%$, still $1.7\times$ below PPO; negative
TTM forecast tokens	Do explicit forecasts help?	8.5% below the modern-v2 baseline; negative
Oracle-LP teacher (SOC waypoints)	First teacher attempt	Given realised future prices; not deployable
Real AEMO dispatch replay (Dalrymple North)	Realistic baseline	Kept as the dispatch-matched baseline
SDP-teacher corpora (v1 full-corpus; v2cal conservative)	Honest distillation	**Shipped policy**

C Provenance and Reproducibility

We release the code, the training corpora, and the model weights, together with a provenance record that maps every headline number in this paper to the artifact and command that produced it.

- **Code and environment.** Source code: `https://github.com/mrvictoru/energydecision`. The evaluation environment is pinned in `paper/environment.txt` (Python 3.10.12, PyTorch 2.9.0+cu130, CUDA 13.0), and all reported results correspond to the `preprint-v1` release tag, at which the test suite (387 tests) passes.
- **Models and data.** The shipped checkpoint (`aemo_dt_sdp_jtsoc_v2cal.pt`) and its training corpus (`dt_trajectories_jtsoc_v2cal_conservative.parquet`) are hosted at `https://huggingface.co/mrvictoru/energydecision-dt-v2-sdp` and `https://huggingface.co/datasets/mrvictoru/AEMO_simulated_trade_sdp`. Reproducing the corpus additionally requires the SOC-waypoint teacher helper `https://huggingface.co/mrvictoru/energydecision-waypoint-dt`.
- **Headline numbers.** Every reported value is traced in `paper/audit/provenance.tsv`; the identity-surface statistics are in `paper/audit/artifacts/stagec_v2cal_extended_significance.json`, and the impact-gate analysis in `paper/audit/artifacts/stagec_v2cal_impact_analysis.json`.
- **Validation.** Train/evaluation temporal separation was checked against the source data windows (`paper/audit/artifacts/leakage_check.txt`); the student's determinism and the baselines' run-to-run variation were measured with a two-run probe; and the reference-rollout cache key was versioned to include model and data content hashes after a stale-cache issue was identified (`paper/audit/artifacts/cache_data_staleness_finding.md`).
- **Provenance caveat.** The shipped `return_scale` (23,284.83) is the correct value for inference but does not reproduce exactly as $\mathrm{p90}(|J_t|)/10$ of the released corpus; the discrepancy is documented in the audit notes above.